

```
#!/usr/bin/env python3
"""AR-Desk 主入口：每个交易日美东 09:15-09:30 之间跑一次，产出盘前简报。
```

用法：

```
python main.py          # 正常运行（带时间窗口保护）
python main.py --force   # 忽略时间窗口，随时测试
python main.py --date 2026-09-09  # 复盘指定日期
python main.py --stats    # 打印日型统计
"""

from __future__ import annotations

import argparse
import json
import os
from datetime import datetime
from zoneinfo import ZoneInfo

import yaml

from src import brief, data as D, engine, gex as G, journal, risk

ET = ZoneInfo("America/New_York")

def in_window(now: datetime) -> bool:
    return now.weekday() < 5 and "09:05" <= now.strftime("%H:%M") <= "09:35"

def main() -> int:
    ap = argparse.ArgumentParser()
    ap.add_argument("--force", action="store_true")
    ap.add_argument("--date")
    ap.add_argument("--stats", action="store_true")
    ap.add_argument("--config", default="config.yaml")
    args = ap.parse_args()

    with open(args.config, encoding="utf-8") as f:
        cfg = yaml.safe_load(f)

    if args.stats:
        print(journal.stats(cfg["output"]["journal"]))
        return 0

    now = datetime.now(ET)
```

```

if not args.force and not args.date and not in_window(now):
    print(f"[skip] 当前美东时间 {now:%Y-%m-%d %H:%M}, 不在 09:05-09:35 运行窗口内")
    return 0

today = datetime.strptime(args.date, "%Y-%m-%d").replace(tzinfo=ET) if args.d

# --- 数据 ---
bars5 = D.get_bars(cfg["symbol"]["futures"], "5m", "10d")
bars1h = D.get_bars(cfg["symbol"]["futures"], "1h", "60d")
daily = D.get_daily(cfg["symbol"]["futures"])
atr20 = D.atr(daily, 20)
# 参考价 = 目标日 09:30 ET 之前的最后一个收盘（实盘就是开盘前最新价，复盘也一致）
cutoff = today.replace(hour=9, minute=30, second=0, microsecond=0)
pre = bars5[bars5.index < cutoff]
px = float((pre if not pre.empty else bars5)["close"].iloc[-1])

# --- 引擎 ---
sess = engine.build_sessions(bars5, today, cfg)
day = engine.classify(sess, atr20, cfg)
gx = G.load(cfg, px)
ladder = engine.target_ladder(sess, bars1h, px, gx, day["bias"]) if sess.get(
zone = engine.entry_zone(sess, day["bias"], cfg)

rstate = risk.read_state(cfg["output"]["journal"], today.strftime("%Y-%m-%d"))
gates = engine.gates(day, sess, zone, ladder, gx, px, cfg, rstate, atr20)
v, fails = engine.verdict(gates)

pos = None
if zone:
    entry = (zone["zone_low"] + zone["zone_high"]) / 2
    pos = risk.size_position(entry, zone["invalidation"], cfg)

note_key = "positive_gamma_note" if gx.get("regime") == "positive" else "nega
ctx = {
    "date": today.strftime("%Y-%m-%d"), "price": px, "atr20": round(atr20, 1)
    "sessions": sess, "day": day, "gex": gx, "gex_note": cfg["gex"].get(note_
    "ladder": ladder, "zone": zone, "gates": gates, "verdict": v, "fails": fa
    "position": pos, "risk_state": rstate, "cfg": cfg,
}

# --- 输出 ---
outdir = cfg["output"]["dir"]
os.makedirs(outdir, exist_ok=True)
md = brief.render(ctx)
stem = f"{outdir}/brief_{ctx['date']}"
with open(f"{stem}.md", "w", encoding="utf-8") as f:
    f.write(md)

```

```
with open(f"{stem}.json", "w", encoding="utf-8") as f:
    json.dump({k: val for k, val in ctx.items() if k != "cfg"}, f,
               ensure_ascii=False, indent=2, default=str)
print(md)

journal.log_plan(cfg["output"]["journal"], ctx)
if cfg["output"].get("telegram"):
    journal.telegram(md)
return 0

if __name__ == "__main__":
    raise SystemExit(main())
```